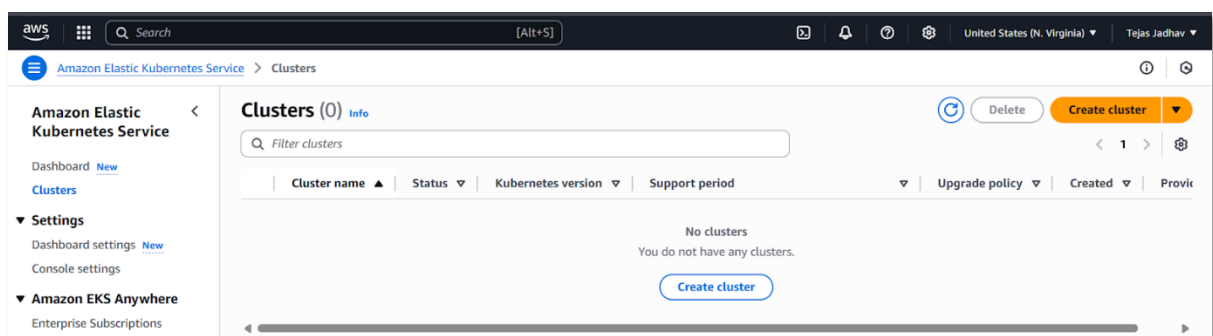


Setting Up an EKS Cluster on AWS and Exposing an Application

Introduction:

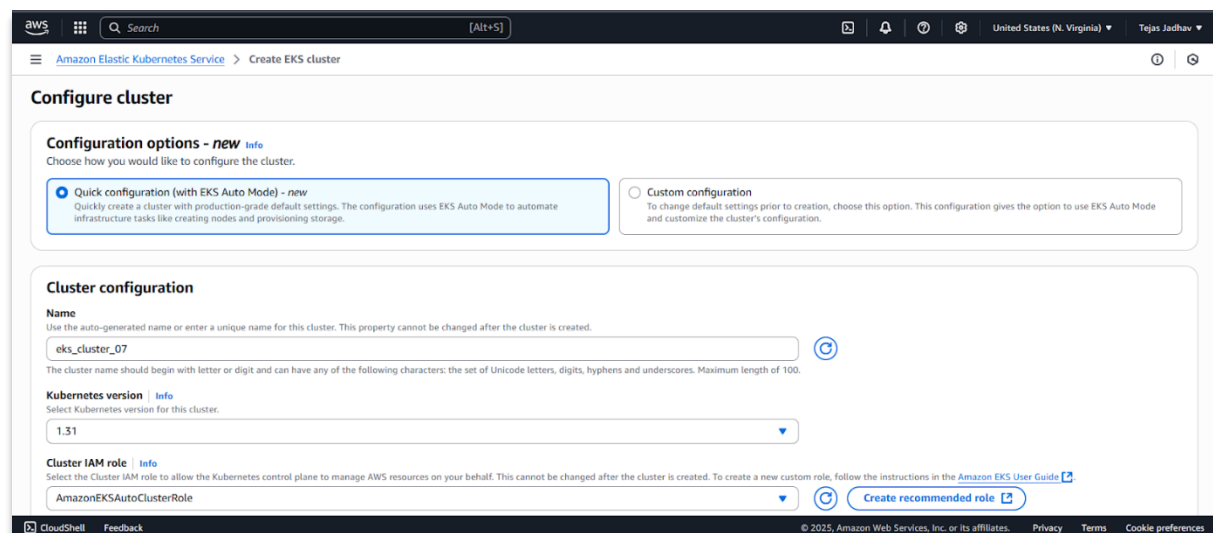
This document provides a clear, step-by-step guide to setting up an Amazon EKS (Elastic Kubernetes Service) cluster on AWS. It walks through deploying a sample application using NGINX and exposing it to the internet via a LoadBalancer. The guide ensures all required configurations, networking, and security settings are properly configured, enabling users to efficiently deploy and manage scalable, highly available Kubernetes workloads on AWS.

Step 1 : Navigate to the EKS Console and Initiate Cluster Creation.



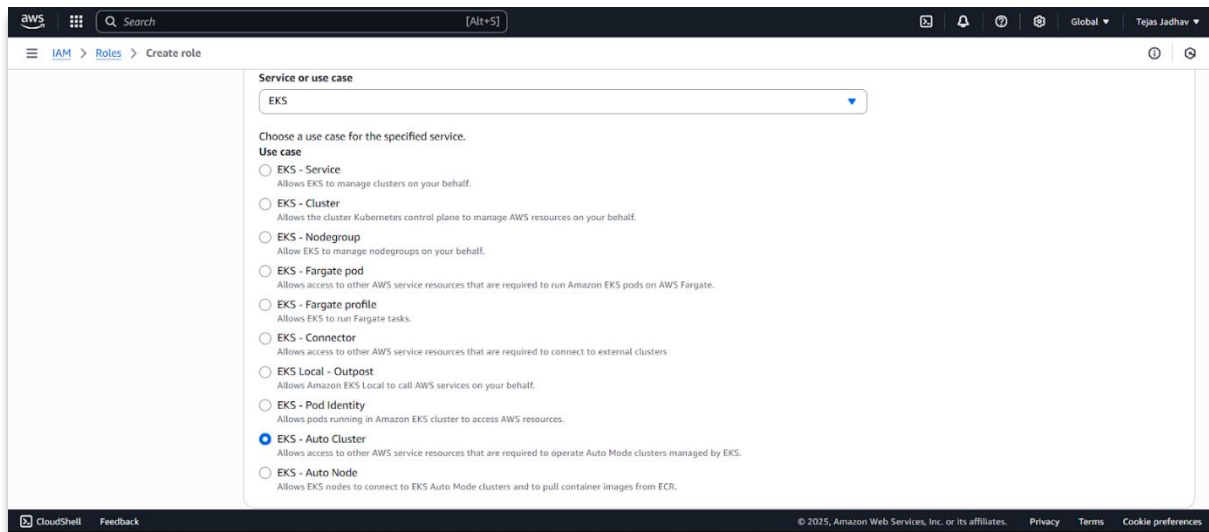
Log in to the **AWS Management Console**, navigate to the Amazon EKS dashboard, and click on **Create Cluster** to begin setting up a new EKS cluster.

Step 2: Provide a Cluster Name and Configure IAM Role



In the Cluster Name field, enter a descriptive name for your cluster, such as **"eks_cluster_07"**. Then, under the Cluster Service Role section, assign an appropriate IAM role to the cluster. Ensure that this IAM role has the necessary permissions, such as the **AmazonEKSClusterPolicy**, to allow EKS to manage AWS resources on your behalf.

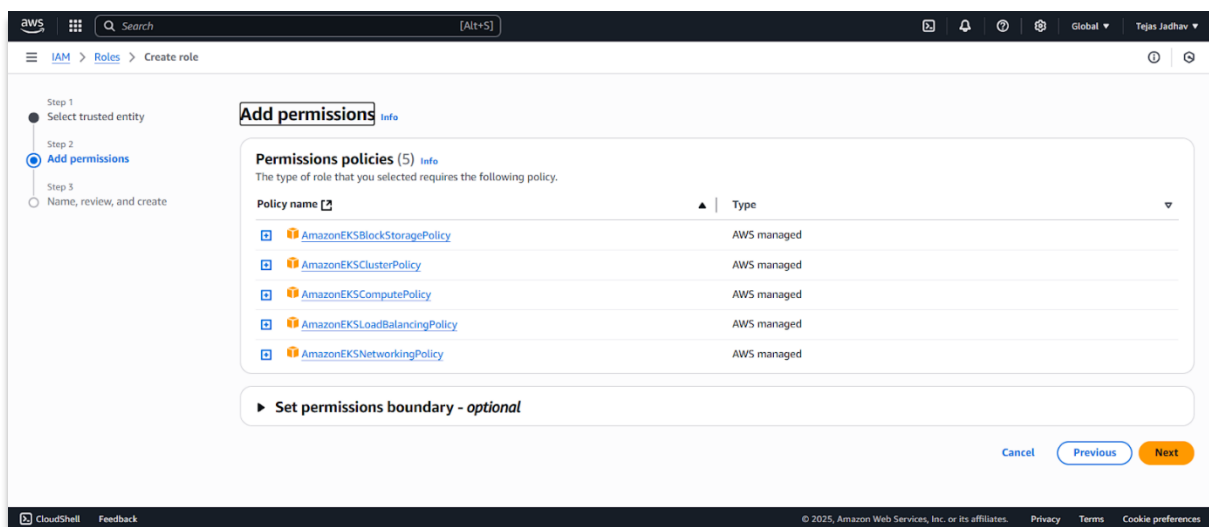
Step 3 : Configure the IAM Role



In the **Configure IAM Role** section, select the default EKS service role provided by AWS.

For the Use Case, choose the default option "**EKS - Cluster Auto**".

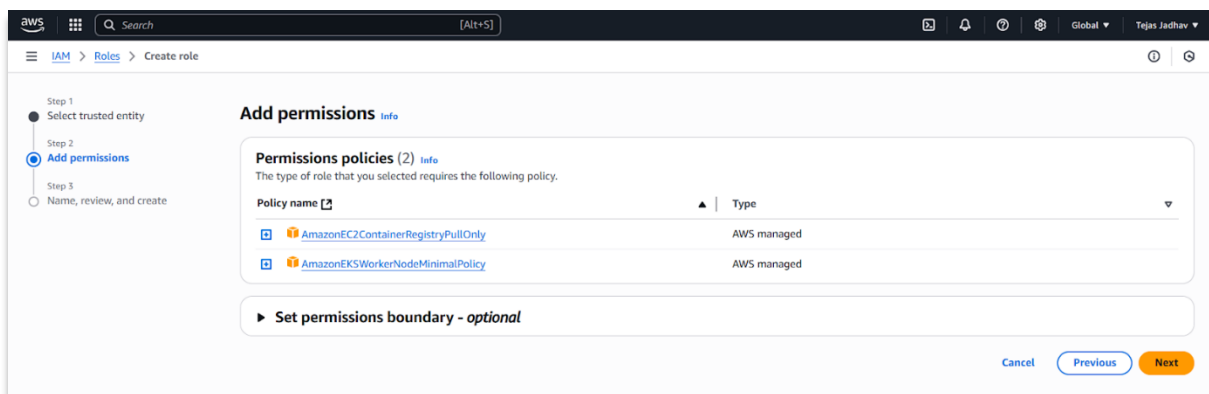
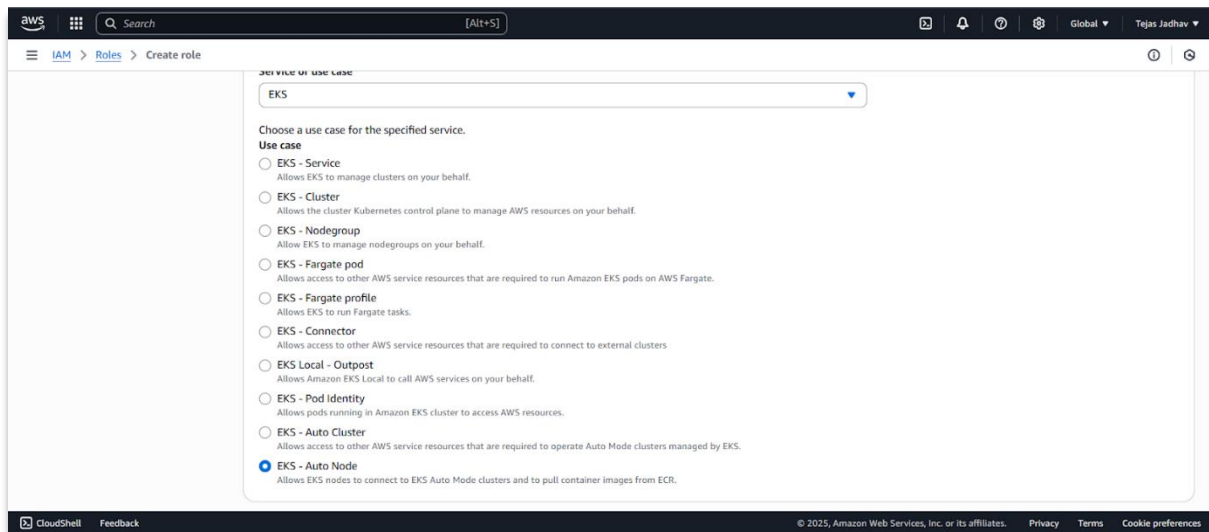
This ensures that the EKS service has the necessary permissions to manage the resources associated with your cluster



After **configuring the IAM** role, AWS automatically attaches the required policies to enable EKS operations.

These policies provide the cluster with permissions to manage essential resources such as compute, storage, networking, and load balancing.

Step 4 : Configure EKS Node Role



In this step, select the Default Service as EKS and the Use Case as **EKS - Auto Node**.

AWS will automatically attach the following policies to the IAM role:

- **EC2ContainerRegistryReadOnly**: Allows the nodes to pull container images from Amazon Elastic Container Registry (ECR).
- **EKSWorkerNodeMinimalPolicy**: Grants the minimal set of permissions needed for EKS worker nodes to operate effectively

Step 5 : Finalize Configuration and Create the Cluster

The screenshot shows the 'Create EKS cluster' page in the AWS Management Console. The page is titled 'Amazon Elastic Kubernetes Service > Create EKS cluster'. It contains several sections for configuration:

- Cluster IAM role:** Select the Cluster IAM role to allow the Kubernetes control plane to manage AWS resources on your behalf. This cannot be changed after the cluster is created. To create a new custom role, follow the instructions in the [Amazon EKS User Guide](#). The selected role is `AmazonEKSAutoClusterRole`. A button 'Create recommended role' is available.
- Node IAM role:** Nodes need an EC2 Instance IAM Role to launch and register with a cluster. To create a new custom role, follow the instructions in the [Amazon EKS User Guide](#). The selected role is `AmazonEKSAutoNodeRole`. A button 'Create recommended role' is available.
- VPC:** Select a VPC to use for your EKS cluster resources. The selected VPC is `vpc-0175d7558c52a4ac | Default`. A button 'Create VPC' is available.
- Subnets:** Choose the subnets in your VPC where the control plane may place elastic network interfaces (ENIs) to facilitate communication with your cluster. To create a new subnet, go to the corresponding page in the [VPC console](#). The selected subnets are:
 - `subnet-092463e180b802ea7` (us-east-1a, 172.31.0.0/20, Type: Public)
 - `subnet-040330e4147815d46` (us-east-1f, 172.31.64.0/20, Type: Public)
 - `subnet-0f9c2cc5673ae3a1c` (us-east-1c, 172.31.16.0/20, Type: Public)
 - `subnet-0bc2f7e05092f2985` (us-east-1b, 172.31.80.0/20, Type: Public)
 - `subnet-091c90e6d88bc777e` (us-east-1d, 172.31.32.0/20, Type: Public)A button 'Clear selected subnets' is available.

At the bottom right, there are 'Cancel' and 'Create' buttons.

Leave all other parameters at their default settings. The **VPC** and **Subnets** will be automatically selected based on your AWS environment.

Review the entire configuration, and once everything looks correct, click **Create** to start the cluster creation process.

AWS will begin provisioning the EKS cluster, which may take several minutes to complete.

Step 6 : Monitor Cluster Creation Status .

The screenshot shows the 'eks_cluster_07' page in the AWS Management Console. The page is titled 'Amazon Elastic Kubernetes Service > Clusters > eks_cluster_07'. It contains the following information:

- Cluster info:**
 - Status:** Creating
 - Kubernetes version:** 1.31
 - Support period:** Standard support until November 26, 2025
 - Provider:** EKS
 - Cluster health:** 0 (green)
 - Upgrade insights:** 0 (green)
 - Node health issues:** 0 (green)

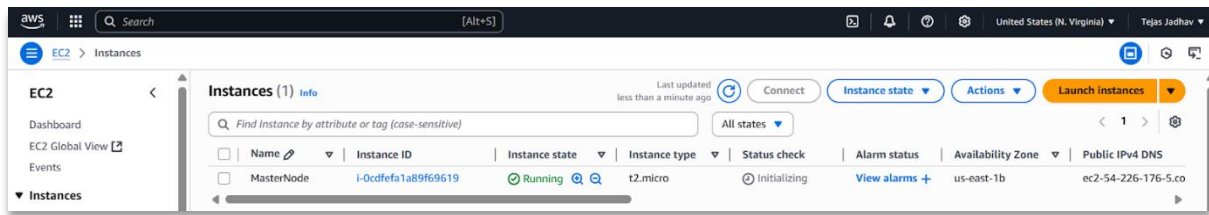
At the top right, there are 'Delete cluster' and 'Monitor cluster' buttons. A message at the top states: 'The Amazon EKS console does not show Kubernetes resources until the cluster has an ACTIVE or UPDATING state. Please try again in a few minutes.'

After clicking the **Create button**, you will be redirected to the **EKS Dashboard**, where the cluster status will appear as **Creating**.

This process may take several minutes as AWS provisions the necessary resources and sets up the cluster infrastructure.

Once the setup is complete, the status will update to **Active**, indicating that the cluster is **ready for use**.

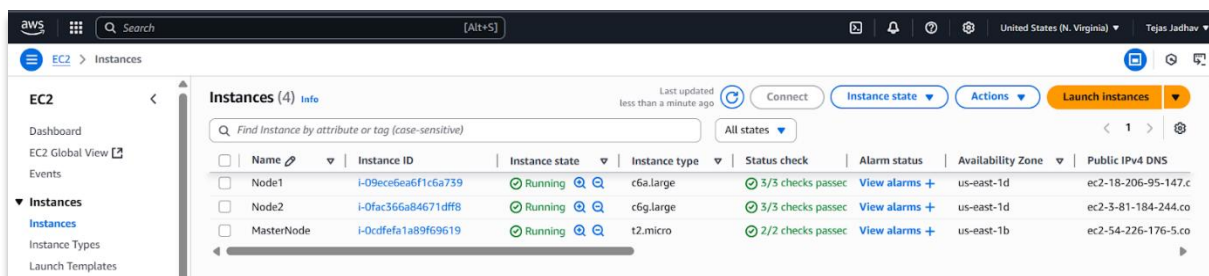
Step 7 : Create an Instance for the Cluster Master.



To manage your **EKS cluster**, launch an **EC2** instance named **inst_master** with minimal configuration.

This instance will act as the cluster master node for performing administrative tasks.

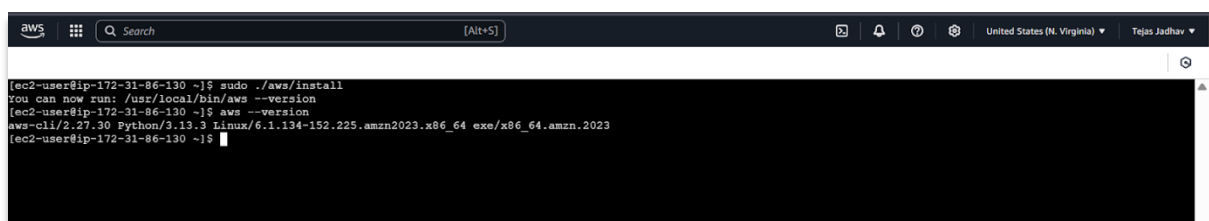
Step 8 : Verify Node Instances Creation



After applying the cluster configuration, go to the **EC2 Console** where you will see two additional instances created automatically.

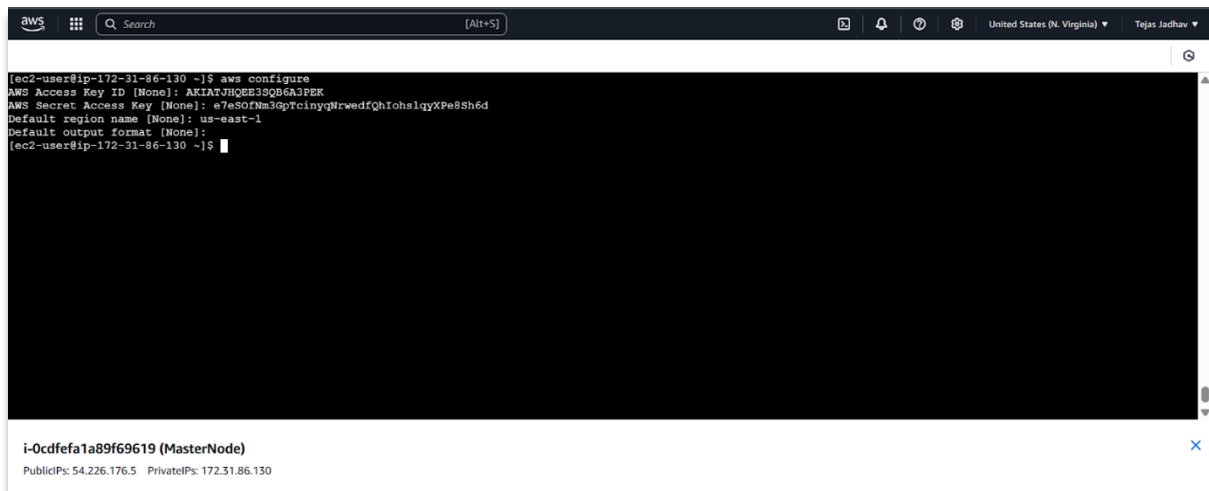
These instances are the worker nodes of your **EKS cluster**. They are provisioned during cluster setup and configured to run workloads while communicating with the **master node**.

Step 9 : Install AWS CLI on the Master Instance .



After successfully connecting to the **inst_master** EC2 instance, install the AWS Command Line Interface (CLI) to manage AWS services directly from the terminal.

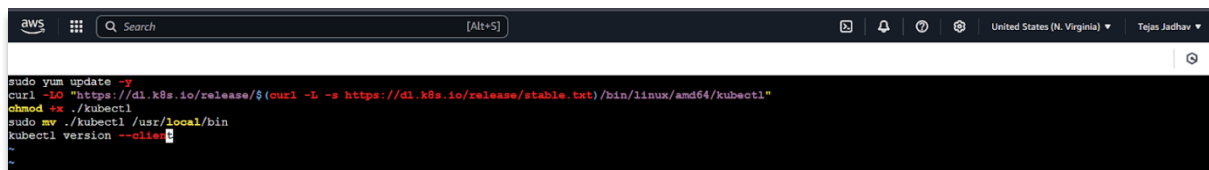
Step 10 : Configure AWS CLI with aws configure

A screenshot of a terminal window on an AWS EC2 instance. The terminal shows the command 'aws configure' being executed. The output prompts for the AWS Access Key ID, Secret Access Key, Default region name, and Default output format. The user has entered 'us-east-1' for the region. The terminal window has a title bar with 'United States (N. Virginia)' and 'Tejas Jadhav'.

```
ec2-user@ip-172-31-86-130 ~]$ aws configure
AWS Access Key ID [None]: AKIATJHQB3SQB6A3PEK
AWS Secret Access Key [None]: e7eSOfNm3GpTcinyqWrwedfQhIohslqyXPe8Sh6d
Default region name [None]: us-east-1
Default output format [None]:
ec2-user@ip-172-31-86-130 ~]$
```

Once the **AWS CLI** is configured, verify the setup by executing a basic **AWS CLI** command.

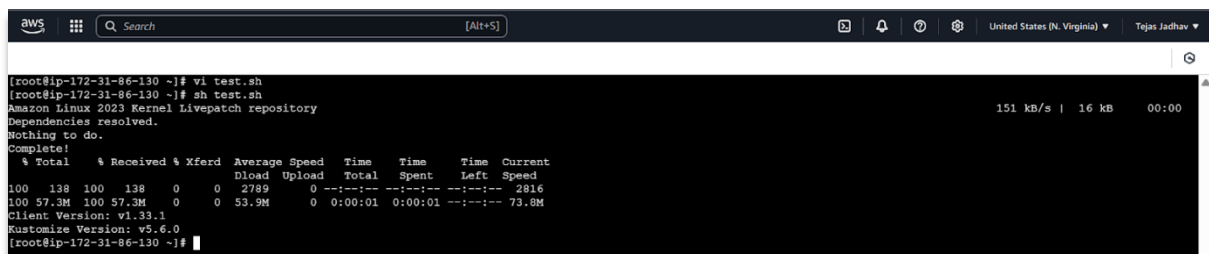
Step 11 : Create a Script to Install kubectl for Kubernetes Commands.

A screenshot of a terminal window showing the installation of kubectl. The user runs 'sudo yum update -y', then 'curl -LO https://dl.k8s.io/release/\$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl', then 'chmod +x ./kubectl', and finally 'sudo mv ./kubectl /usr/local/bin'. The output shows 'kubectl version --client' returning 'v1.30.0'.

```
sudo yum update -y
curl -LO https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl
chmod +x ./kubectl
sudo mv ./kubectl /usr/local/bin
kubectl version --client
v1.30.0
```

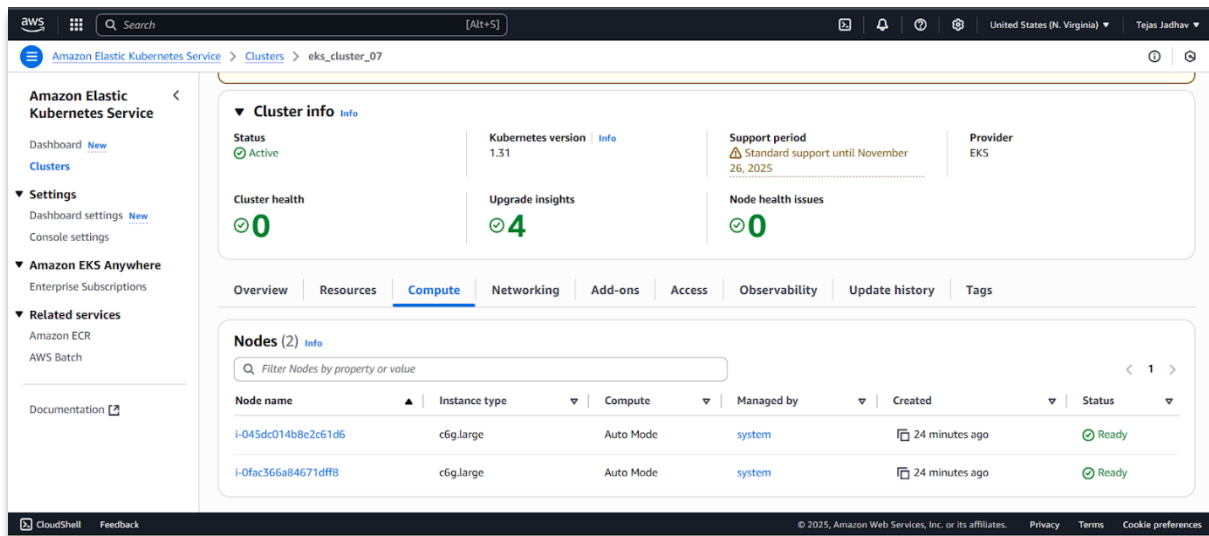
To manage your **EKS cluster** with Kubernetes commands, **install the kubectl** command-line tool.

Step 12 : Execute the Script to Install kubectl and Verify the Version .

A screenshot of a terminal window showing the installation of kubectl and its verification. The user runs 'vi test.sh', then 'sh test.sh'. The script installs kubectl and prints its version. The output shows 'kubectl version --client' returning 'v1.30.0'.

```
(root@ip-172-31-86-130 ~)# vi test.sh
(root@ip-172-31-86-130 ~)# sh test.sh
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 138 100 138 0 0 2789 0 --:--:-- --:--:-- --:--:-- 2816
100 57.3M 100 57.3M 0 0 53.9M 0 0:00:01 0:00:01 --:--:-- 73.8M
Client Version: v1.30.1
Kustomize Version: v5.6.0
(root@ip-172-31-86-130 ~)#
```

Step 13 : Verify Cluster and Node Status .



The output will display the nodes in a **Ready** state, confirming that the cluster is fully operational.

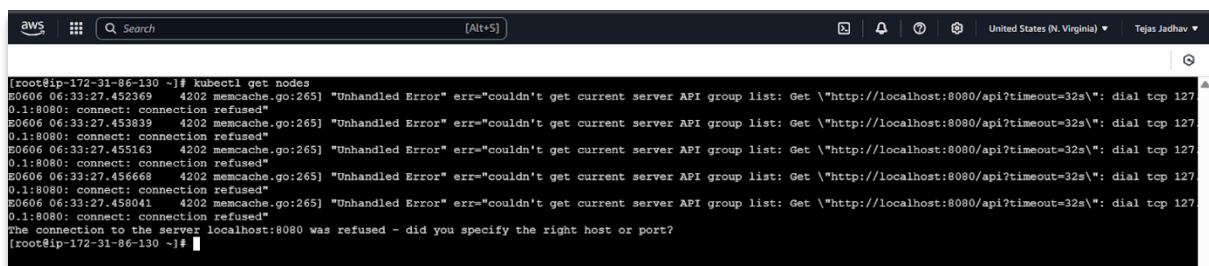
Under the **Nodes** section, the default worker nodes will be listed with their status shown as **Ready**, indicating they are successfully registered and functioning within the EKS cluster.

Step 14 : Connect the Master Node to the EKS Cluster .



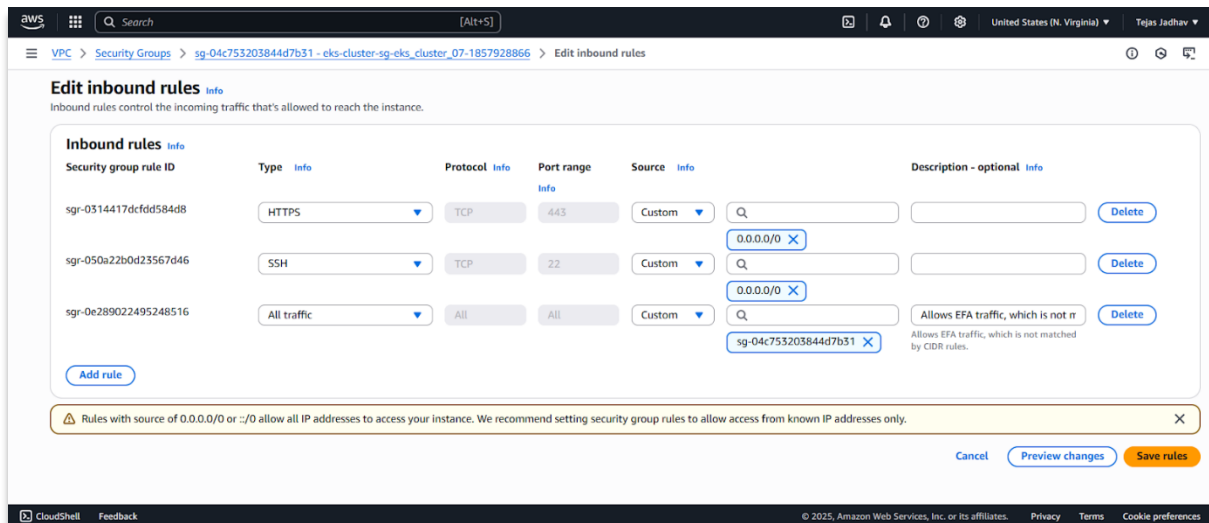
To connect your **inst_master** EC2 instance to the newly created EKS cluster, run the following AWS CLI command to update the kubeconfig file. This enables **kubectl** to communicate with your cluster: `aws eks update-kubeconfig --region <region-name> --name <cluster-name>`

Step 15 : Troubleshoot Connection Issues with kubectl



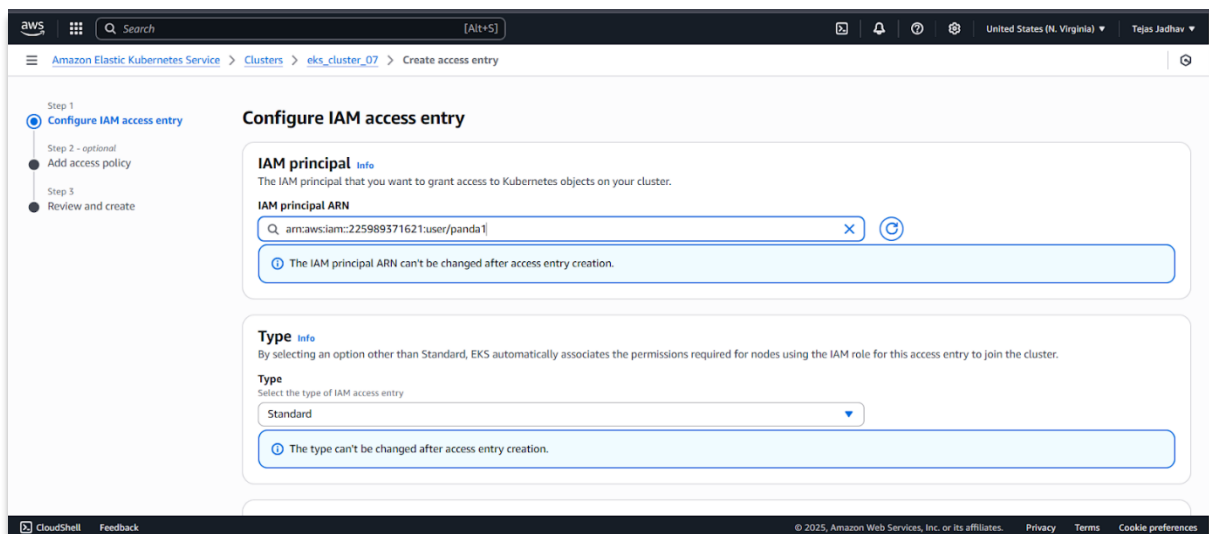
After updating the kubeconfig and trying to connect to the EKS cluster, you might encounter an error.

Step 16 : Resolve Connection Refused Error by Modifying Security Group .

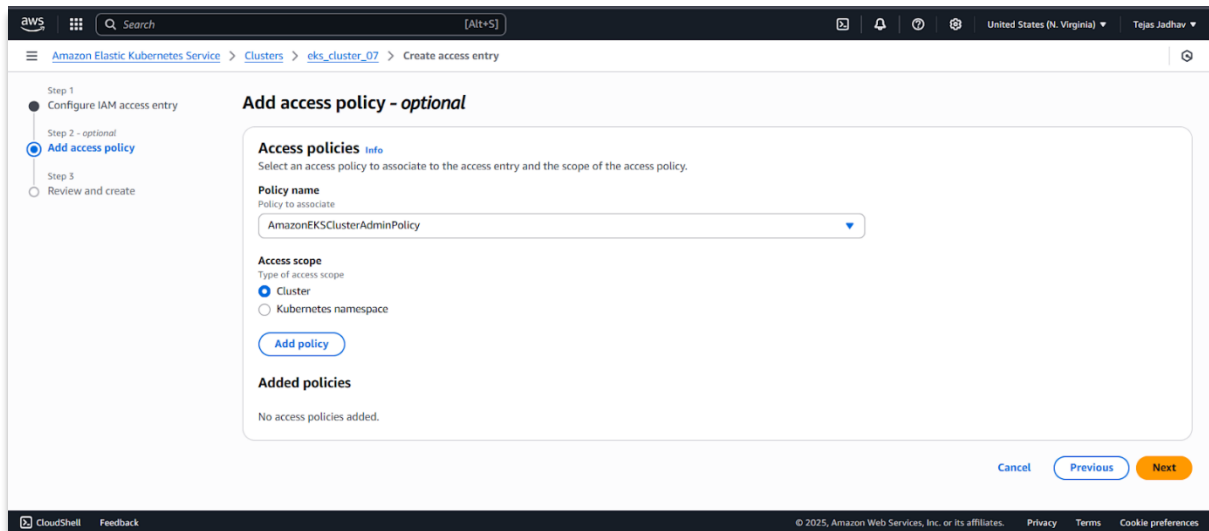


To resolve the "**connection refused**" error, make sure the security group attached to your EKS cluster permits inbound **HTTPS** traffic on **port 443**, allowing communication with the cluster endpoint.

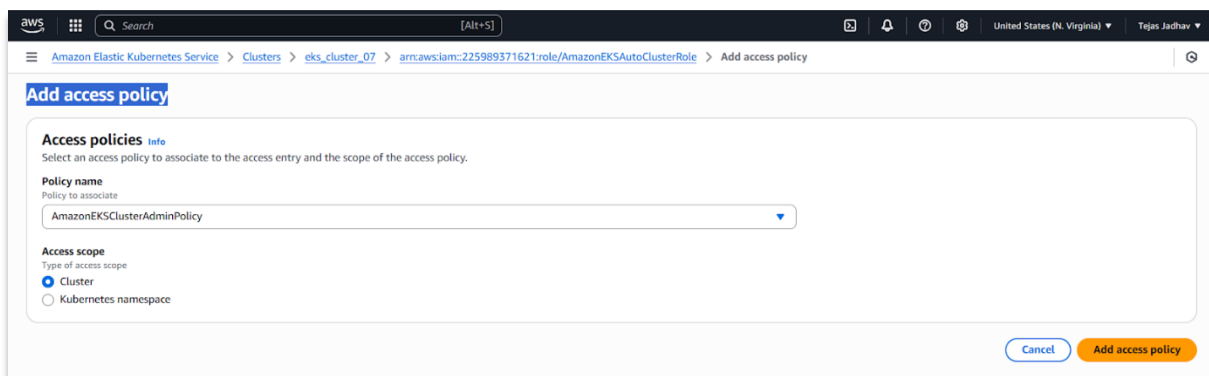
Step 17 : Add IAM User Access Entry to the Cluster.



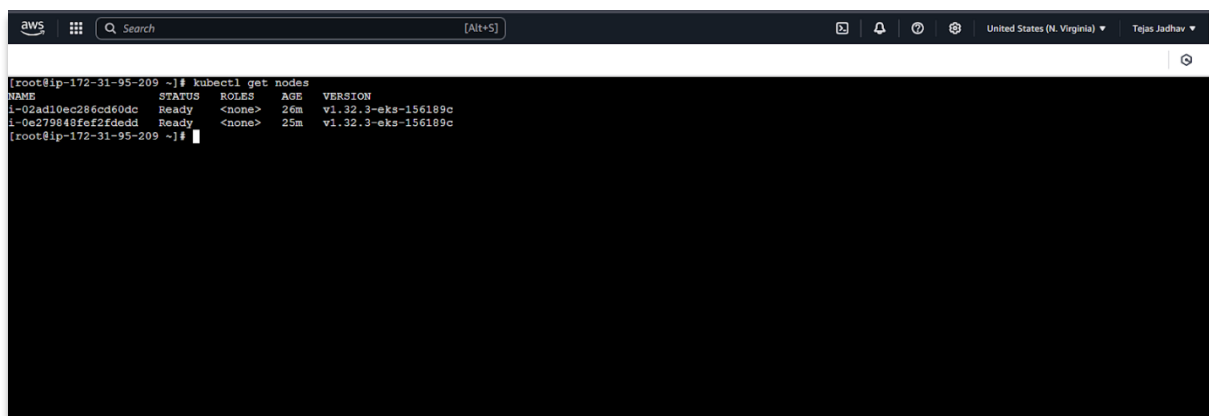
To grant the IAM user the necessary permissions to access the EKS cluster, add the IAM user's ARN to the cluster's access control by updating the aws-auth ConfigMap.



Step 18 : Add the Same Entry Again in the Add access Policy.



Step 19 : Retry the Command and Verify Access .



This confirms that the issue is resolved and the IAM user now has proper access to the EKS cluster.

Step 20 : Create a Deployment Using the nginx Image.

```
[root@ip-172-31-95-209 ~]# kubectl create deployment myapp --image nginx
deployment.apps/myapp created
[root@ip-172-31-95-209 ~]# kubectl get deployment
error: unknown command "gte" for "kubectl"

Did you mean this?
  get
[root@ip-172-31-95-209 ~]# kubectl get deployment
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
myapp     0/1     1            0           24s
[root@ip-172-31-95-209 ~]# kubectl get pod
NAME      READY   STATUS             RESTARTS   AGE
myapp-5b5df85c44-ckb27  0/1     ContainerCreating  0           36s
[root@ip-172-31-95-209 ~]# kubectl expose deployment myapp --port 80
service/myapp exposed
[root@ip-172-31-95-209 ~]# kubectl get svc
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes  ClusterIP   10.100.0.1    <none>         443/TCP   72m
myapp      ClusterIP   10.100.237.105 <none>         80/TCP    7s
[root@ip-172-31-95-209 ~]# kubectl get svc -w
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes  ClusterIP   10.100.0.1    <none>         443/TCP   73m
myapp      ClusterIP   10.100.237.105 <none>         80/TCP    43s
^C[root@ip-172-31-95-209 ~]#
```

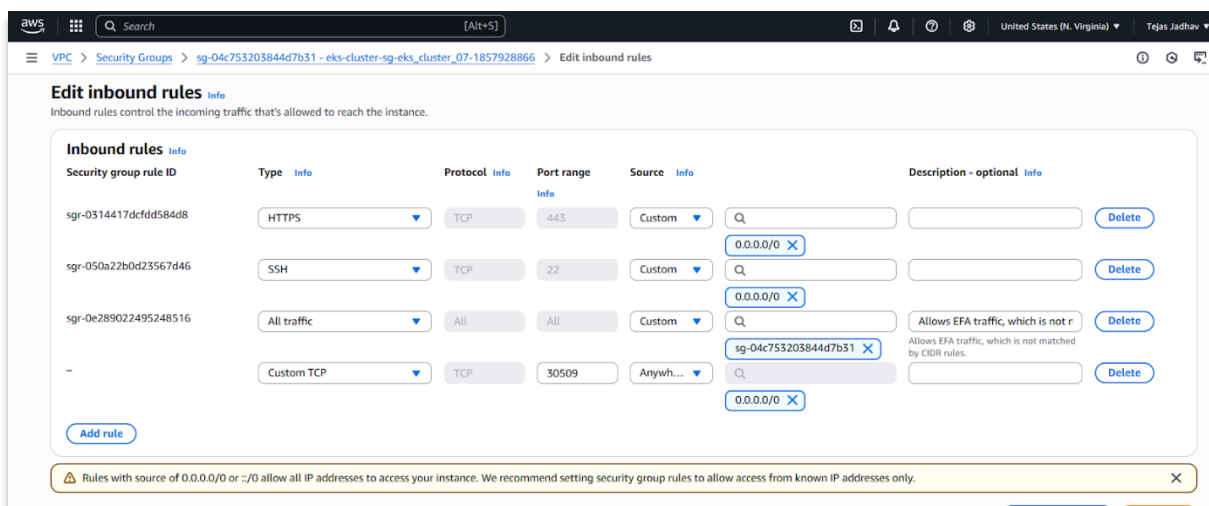
The output should display the pod(s) created by the nginx deployment with a **Running** status, confirming that the deployment was successful.

Step 21 : Expose the Deployment by Creating a Service and Editing with LoadBalancer.

```
[root@ip-172-31-95-209 ~]# kubectl edit svc myapp
service/myapp edited
[root@ip-172-31-95-209 ~]# kubectl get svc -w
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes  ClusterIP   10.100.0.1    <none>         443/TCP   75m
myapp      NodePort    10.100.237.105 <none>         80:30509/TCP 2m44s
^C[root@ip-172-31-95-209 ~]#
```

To make the nginx deployment accessible externally, expose it by creating a Kubernetes Service of type **LoadBalancer**.

Step 22 : Add LoadBalancer Port in the Cluster Security Group to Allow External Access.



To enable external traffic to reach the LoadBalancer and access the nginx service, update the security group associated with your EKS cluster to allow inbound traffic on the required port (usually port 80 for HTTP).

Step 23 : Access the Service Externally Using the LoadBalancer IP.

The screenshot shows a web browser window at the top with the URL `184.73.77.122:30509`. The page displays the "Welcome to nginx!" message, indicating the nginx web server is successfully installed and working. Below the message, it provides links to nginx.org for documentation and nginx.com for commercial support.

Below the browser window, a terminal window shows the following commands and output:

```
[root@ip-172-31-95-209 ~]# kubectl get pod -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATES
myapp-5b5df85c44-ckb27             1/1     Running   0           9m6s  172.31.27.160   i-01f0558383d1f449f               <none>            <none>

[root@ip-172-31-95-209 ~]# kubectl get node -o wide
NAME                                STATUS   ROLES    AGE   VERSION          INTERNAL-IP   EXTERNAL-IP   OS-IMAGE                                     KERNEL-VERSION   CONTAINER-RUNTIME
i-01f0558383d1f449f               Ready    <none>    9m15s  v1.32.3-eks-156189c  172.31.30.51  3.85.229.25   Bottlerocket (EKS Auto) 2025.5.31 (aws-k8s-1.32) 6.1.134           containerd
erd:/1.7.27+bottlerocket          Ready    <none>    42m    v1.32.3-eks-156189c  172.31.88.82  184.73.77.122 Bottlerocket (EKS Auto) 2025.5.31 (aws-k8s-1.32) 6.1.134           containerd
i-02ad10ec286cd60dc               Ready    <none>    42m    v1.32.3-eks-156189c  172.31.88.82  184.73.77.122 Bottlerocket (EKS Auto) 2025.5.31 (aws-k8s-1.32) 6.1.134           containerd
erd:/1.7.27+bottlerocket          Ready    <none>    41m    v1.32.3-eks-156189c  172.31.29.14  3.87.80.214   Bottlerocket (EKS Auto) 2025.5.31 (aws-k8s-1.32) 6.1.134           containerd
i-0e279848fef2fdeedd             Ready    <none>    41m    v1.32.3-eks-156189c  172.31.29.14  3.87.80.214   Bottlerocket (EKS Auto) 2025.5.31 (aws-k8s-1.32) 6.1.134           containerd
erd:/1.7.27+bottlerocket

[root@ip-172-31-95-209 ~]# kubectl get svc -o wide
NAME                                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE   SELECTOR
kubernetes  ClusterIP   10.100.0.1    <none>         443/TCP          80m    <none>
myapp       NodePort    10.100.237.105 <none>         80:30509/TCP     7m59s  app=myapp

[root@ip-172-31-95-209 ~]# kubectl get deployment -o wide
NAME    READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES   SELECTOR
myapp   1/1     1             1           10m    nginx        nginx    app=myapp

[root@ip-172-31-95-209 ~]#
```

Once the LoadBalancer is provisioned and the security group is configured to allow traffic, you can access the nginx service externally using the LoadBalancer's public IP or DNS name.